🎓 Exercício Guiado – Lista de Tarefas (Com Partes do Código)

⏰ Tempo: 4 horas
🎯 Objetivo: Criar um app que adiciona e remove tarefas usando `State`.

🟢 PARTE 1 – Estrutura Inicial

◇ Passo 1 – Criar o main()

Complete o código abaixo:

```
import 'package:flutter/material.dart';

void main() {
  runApp(__________);
}
```

👉 Pergunta:

- O que precisa ser passado dentro do `runApp`?

◇ Passo 2 – Criar o MyApp

Complete:

```
class MyApp extends __________ {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: __________,
    );
  }
}
```

👉 Perguntas:

- Esse widget deve ser Stateless ou Stateful?

- Por quê?

🟢 PARTE 2 – Criando a Tela com Estado

◇ Passo 3 – Criar a TodoPage

Complete:

```
class TodoPage extends __________ {
  const TodoPage({super.key});

  @override
  State<TodoPage> createState() => __________;
}
```

Agora crie a classe State:

```
class _TodoPageState extends State<TodoPage> {

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("______________"),
      ),
      body: const Text("Em construção"),
    );
  }
}
```

👉 Teste até aqui.

🟢 PARTE 3 – Criando o Estado

◇ Passo 4 – Criar a Lista

Complete:

```
final List<String> __________ = [];
```

👉 Pergunta:

- Por que essa variável precisa estar dentro da classe `_TodoPageState`?

◇ Passo 5 – Criar o Controller

Complete:

```
final ___________________ controller = ___________________();
```

👉 Dica:

- Qual classe controla o texto digitado?

🟢 PARTE 4 – Montando o Layout

Substitua o body por:

```
body: Column(
  children: [

    TextField(
      controller: __________,
    ),

    ElevatedButton(
      onPressed: () {},
      child: const Text("Adicionar"),
    ),

  ],
),
```

👉 Teste: O campo e botão devem aparecer.

🟢 PARTE 5 – Criando Função para Adicionar

◇ Passo 6 – Criar função

Complete:

```
void adicionarTarefa() {
  setState(() {
    __________.add(controller.text);
  });

  controller.__________;
}
```

👉 Perguntas:

- O que acontece se remover o setState?
- O que deve ser usado para limpar o campo?

◇ Passo 7 – Conectar o Botão

Complete:

```
ElevatedButton(
  onPressed: __________,
  child: const Text("Adicionar"),
),
```

Teste usando:

```
print(tarefas);
```

🟢 PARTE 6 – Exibindo a Lista

Abaixo do botão, adicione:

```
Expanded(
  child: ListView.builder(
    itemCount: __________.length,
    itemBuilder: (context, index) {
      return ListTile(
        title: Text(__________[index]),
      );
    },
  ),
),
```

👉 Perguntas:

- O que é `index`?
- Por que usamos `.length`?

🟢 PARTE 7 – Removendo Tarefa

◇ Passo 8 – Criar função remover

Complete:

```
void removerTarefa(int index) {
  setState(() {
    __________.removeAt(__________);
  });
}
```

◇ Passo 9 – Adicionar botão de excluir

Complete dentro do `ListTile`:

```
trailing: IconButton(
  icon: const Icon(Icons.delete),
  onPressed: () => __________(index),
),
```

👉 Teste: Adicione 3 tarefas e remova uma.

🧠 Desafios Extras

Complete sozinho:

1. Impedir adicionar tarefa vazia
2. Mostrar quantidade de tarefas no AppBar
3. Permitir adicionar pressionando Enter
4. Mostrar mensagem se a lista estiver vazia